

Runtime Developer Toolkit

FPS Monitor Documentation

Runtime Developer Toolkit

FPS Monitor Documentation

****Package area:**** `RuntimeDeveloperToolkit/PerformanceMonitor`

****Namespace:**** `RuntimePerformanceMonitor`

****Target:**** Unity runtime diagnostics and performance visualization

How to use

Drag and drop the **[FPSMonitor].prefab** into your scene. The prefab is located at RuntimeDeveloperToolkit/PerformanceMonitor/Prefab/[FPSMonitor].prefab

1. Overview

The FPS Monitor is a runtime diagnostics module intended to help developers inspect application performance while a Unity application is running.

The current implementation provides:

- Current FPS
- Current frame time
- Average FPS
- Minimum and maximum FPS
- Low 1% FPS
- Low 10% FPS
- FPS history storage
- FPS and frame-time graphs
- Rendering statistics
- Device and system information
- Application and quality settings information
- A demo stress-test utility that instantiates and destroys prefabs

The system is divided into four main layers:

1. ****Bootstrap / orchestration**** – starts and updates the monitoring system.
2. ****Core calculators**** – calculate FPS, rendering, system, and application data.
3. ****Data structures**** – hold snapshots and statistics.
4. ****UI views**** – display the collected data.

2. Folder Structure

RuntimeDeveloperToolkit/

└─ PerformanceMonitor/

│ └─ Demo/

│ │ └─ Prefab/

│ │ │ └─ Box.prefab

│ │ └─ Scenes/

│ │ │ └─ DemoScene.unity

│ │ └─ Script/

│ │ │ └─ StressTestManager.cs

│

│ └─ Prefab/

│ │ └─ [FPSMonitor].prefab

│

│ └─ Scripts/

│ │ └─ Bootstrap/

│ │ │ └─ PerformanceMonitor.cs

│ │

│ │ └─ Core/

│ │ │ └─ AppStats/

│ │ │ │ └─ AppStatsCalculator.cs

│ │ │ └─ FPS/

│ │ │ │ └─ FPSCalculator.cs

│ │ │ │ └─ FPSHistory.cs

│ │ │ │ └─ FPSRate.cs

│ │ │ └─ RenderingStats/

│ │ │ │ └─ RenderingInfoCalculator.cs

│ │ │ └─ SystemStats/

```
| | └─ SystemStatsCalculator.cs
| |
| └─ Data/
| | └─ AppStats.cs
| | └─ FPSSnapshot.cs
| | └─ FPSStats.cs
| | └─ RendererStats.cs
| | └─ SystemStats.cs
| |
| └─ UI/
| | └─ ApplInfoView.cs
| | └─ FPSView.cs
| | └─ Graph.cs
| | └─ GraphView.cs
| | └─ MonitorCanvasTabButton.cs
| | └─ PerformanceMonitorCanvas.cs
| | └─ RenderingInfoView.cs
| | └─ SystemInfoView.cs
| |
| └─ Utils/
|   └─ RDT_TextFormator.cs
|
└─ Textures/
  └─ hidden.png
    └─ show.png
---
```

3. Runtime Architecture

3.1 `PerformanceMonitor`

`PerformanceMonitor` is the main coordinator.

Responsibilities:

- Maintains a singleton instance.
- Persists the monitor object across scene loads.
- Creates calculator and history objects.
- Initializes the UI.
- Starts monitoring after a short real-time delay.
- Updates FPS every frame.
- Updates rendering statistics once per second.
- Sends calculated data to the relevant UI views.

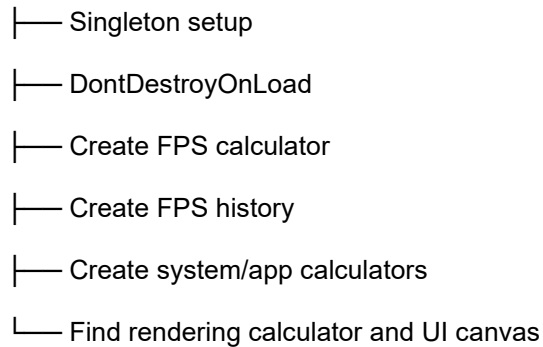
The class uses:

```
public static PerformanceMonitor Instance;
```

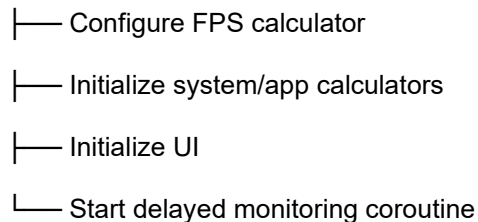
The monitor is initialized in `Awake()`, while runtime monitoring begins from a coroutine after `StartDelay`.

Lifecycle

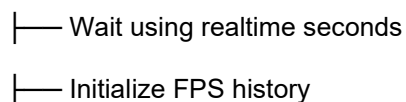
Awake



Start



StartFpsMonitor



- |— Initialize rendering calculator
- |— Populate system information
- |— Populate application information
- └— Enable monitoring

Update

- |— Monitor FPS
- └— Monitor rendering statistics

Configuration currently defined in `PerformanceMonitor`

Field	Current value	Purpose
--- ---:---		
`fpsRate`	`Hz_25`	FPS calculation frequency
`HistorySize`	`300`	Number of snapshots retained
`StartDelay`	`0.1` seconds	Initialization delay
`Delay_RenderingStats`	`1.0` second	Rendering statistics refresh interval

`HistorySize` is clamped to a minimum of 50 during initialization.

4. FPS Calculation

4.1 `FPSRate`

The `FPSRate` enum defines the calculation mode:

```
public enum FPSRate
{
    EveryFrame,
    Hz_1,
    Hz_5,
    Hz_10,
    Hz_25,
    Hz_60
}

| Mode | Sampling interval |
```

```

|---|---:|

| `EveryFrame` | Every `Update` call |

| `Hz_1` | 1 second |

| `Hz_5` | 0.2 seconds |

| `Hz_10` | 0.1 seconds |

| `Hz_25` | 0.04 seconds |

| `Hz_60` | Approximately 0.0167 seconds |

```

4.2 `FPSCalculator`

`FPSCalculator` supports two calculation strategies.

Every-frame mode

The current frame time is used directly:

$FPS = 1 / frameTime$

$FrameTime = frameTime$

This represents instantaneous FPS for the current frame.

The calculator rejects non-positive frame times.

Accumulated mode

For lower sampling frequencies, the calculator accumulates:

- Total frame time
- Number of frames

When the configured interval is reached:

$Average\ FPS = accumulatedFrames / accumulatedFrameTime$

$Average\ Frame\ Time = accumulatedFrameTime / accumulatedFrames$

The accumulated counters are reset after a snapshot is produced.

`FPSSnapshot`

```
public struct FPSSnapshot
```

```

{
    public float FPS;

    public float FrameTime;
}

```

`FrameTime` is stored in seconds. The UI converts it to milliseconds by multiplying it by `1000`.

5. FPS History and Statistics

5.1 `FPSHistory`

`FPSHistory` stores snapshots in a circular buffer.

Internal state:

- `fpsSnapshotHistory` – active circular buffer
- `sortedFpsSnapshotHistory` – reusable sorting buffer reference
- `historySize` – maximum number of snapshots
- `writeIndex` – next write position
- `snapshotCount` – number of valid snapshots currently stored
- `curFpsStats` – latest calculated statistics

Circular buffer behavior

Each new snapshot is written at `writeIndex`. The index is incremented and wrapped to zero when it reaches `historySize`.

`writeIndex` points to the next position that will be written. Therefore:

Newest snapshot = writeIndex - 1

Second newest = writeIndex - 2

The graph and statistics systems reconstruct chronological order using the circular-buffer index.

5.2 Calculated statistics

The system calculates:

- Minimum FPS
- Maximum FPS
- Minimum frame time
- Maximum frame time
- Average FPS
- Low 1% FPS
- Low 10% FPS

Average FPS is calculated using total frame time:

Average FPS = snapshotCount / totalFrameTime

This is preferable to calculating the arithmetic mean of instantaneous FPS values.

5.3 Low 1% and low 10%

The snapshots are sorted by descending frame time, meaning the slowest frames appear first.

The low-percentile FPS values are calculated as an FPS equivalent:

Low percentile FPS =

selected sample count / sum of selected frame times

For small history sizes, the implementation clamps the selected sample count to at least one sample and no more than the current snapshot count.

Important implementation note

The current implementation allocates a new sorting array in `CalcFpsStats()`:

```
sortedFpsSnapshotHistory = new FPSSnapshot[totalSnapShots];
```

This can create managed allocations whenever a snapshot is added. For a production performance tool, consider allocating the sorting buffer once at setup and reusing it.

The current implementation also sorts the samples every time statistics are calculated. A later optimization could calculate low-percentile metrics less frequently than the basic FPS statistics.

6. Rendering Statistics

6.1 `RenderingInfoCalculator`

The rendering calculator uses Unity's `ProfilerRecorder` API for selected render metrics.

Recorded metrics:

- Draw calls
- SetPass calls
- Batches
- Triangles
- Vertices
- Shadow casters

Additional scene counts are collected through Unity APIs:

- Camera count
- Light count
- Renderer count
- Skinned mesh renderer count

Data source summary

Metric	Source
--------	--------

---	---
-----	-----

Draw calls	<code>ProfilerRecorder</code>
------------	-------------------------------

SetPass calls	<code>ProfilerRecorder</code>
---------------	-------------------------------

Batches	<code>ProfilerRecorder</code>
---------	-------------------------------

Triangles	<code>ProfilerRecorder</code>
-----------	-------------------------------

Vertices	<code>ProfilerRecorder</code>
----------	-------------------------------

Shadow casters	<code>ProfilerRecorder</code>
----------------	-------------------------------

```
| Cameras | `Camera.allCamerasCount` |  
| Lights | `FindObjectsByType<Light>()` |  
| Renderers | `FindObjectsByType<Renderer>()` |  
| Skinned meshes | `FindObjectsByType<SkinnedMeshRenderer>()` |
```

The rendering statistics are refreshed approximately once per second by `PerformanceMonitor`.

Performance considerations

The scene enumeration calls can be expensive in large scenes. They are not executed every frame, but they should still be profiled in representative projects.

The `ProfilerRecorder` instances are started in `OnEnable()` and disposed in `OnDisable()`.

7. System Information

7.1 `SystemStatsCalculator`

System information is collected through Unity's `SystemInfo` and `Screen` APIs.

Collected categories:

Device

- Device model
- Device type
- Operating system

CPU

- Processor name/type
- Processor count
- Processor frequency

GPU

- GPU name
- GPU vendor
- Graphics API
- Graphics device version

Memory

- GPU memory size
- System RAM size

Display

- Display width
- Display height
- Current refresh rate

Limitations

Some values are platform-dependent and may return zero, an approximate value, or a generic string.

``SystemInfo.processorFrequency`` is not guaranteed to represent the processor's real-time operating frequency.

``SystemInfo.graphicsDeviceVersion`` should be treated as Unity's reported graphics device version string, not necessarily a reliable standalone GPU driver version.

8. Application Information

8.1 ``AppStatsCalculator``

Application data is captured from Unity's application and quality settings APIs.

Collected values:

- Application name
- Application version
- Unity version
- Scripting backend
- Target platform
- Development build status
- Quality level
- Color space
- VSync count
- Target frame rate

The scripting backend is detected using compilation symbols:

- ``IL2CPP``
- ``Mono``
- ``Unknown``

The target frame rate is displayed as ``Max`` when ``Application.targetFrameRate`` is ``-1``.

VSync display currently supports:

- ``0`` – Off
- ``1`` – VSync every frame
- ``2`` – VSync every two frames

Other VSync values are not explicitly formatted by the current UI implementation.

9. Graph System

9.1 `Graph`

`Graph` inherits from Unity's `Graphic` class and builds its mesh using `VertexHelper`.

Each line segment is rendered as a thin quad made from:

- Four vertices
- Two triangles

The graph supports:

- FPS display
- Frame-time display
- Configurable line thickness
- Circular-buffer history
- Right-to-left data ordering
- Minimum-value padding for uncalculated samples

Graph direction

The newest sample is drawn on the right side.

Left Right

Older samples ----- Newest sample

The graph reconstructs the newest-to-oldest sequence from:

- `historySize`
- `writeIndex`
- `snapshotCount`

Uncalculated samples

The graph draws across the complete `historySize`.

When `snapshotCount` is lower than `historySize`, uncalculated points use `minValue`. This keeps the unused region flat at the graph's minimum value.

Frame-time units

The graph stores frame time in seconds but displays it in milliseconds:

Graph frame time = FrameTime * 1000

The graph receives its minimum and maximum values from `GridView`.

Graph validation

The graph checks:

- Snapshot array availability
- History size
- Snapshot count

- Array capacity
- FPS/frame-time mode selection
- Minimum and maximum range

It rejects the ambiguous case where both FPS and frame time are selected simultaneously.

10. Graph View

10.1 `GraphView`

`GraphView` is responsible for:

- Deciding whether the graph should refresh
- Calculating displayed minimum and maximum values
- Updating min/mid/max labels
- Passing history data to the `Graph` component

Two graph views are used by `PerformanceMonitorCanvas`:

5. FPS graph
6. Frame-time graph

The FPS graph is configured with:

`showFPS = true`

`showFrameTime = false`

The frame-time graph is configured with:

`showFPS = false`

`showFrameTime = true`

Graph range

The current implementation applies:

`Displayed minimum = raw minimum * 0.75`

`Displayed maximum = raw maximum * 1.25`

The same adjusted values are used for the graph labels and graph normalization.

Graph refresh timing

The current graph delay is based on:

`graphUpdateDelay = fpsCalcDelay + 0.75 seconds`

This means the graph refresh interval depends on the FPS calculation rate.

For a future version, it may be preferable to expose a separate graph refresh interval so FPS sampling and graph rendering can be controlled independently.

11. UI System

11.1 `PerformanceMonitorCanvas`

`PerformanceMonitorCanvas` connects the runtime data providers to the UI views.

It manages:

- FPS view
- FPS graph
- Frame-time graph
- Rendering information view
- System information view
- Application information view
- Tab navigation

It initializes the views and routes update calls from `PerformanceMonitor`.

11.2 `FPSView`

Displays:

- Current FPS
- Current frame time
- Maximum FPS
- Minimum FPS
- Average FPS
- Low 1% FPS
- Low 10% FPS

Numerical formatting:

- FPS values: one decimal place
- Frame time: three decimal places in milliseconds

11.3 `RenderingInfoView`

Displays:

- Draw calls
- SetPass calls
- Batches
- Triangles
- Vertices
- Renderers
- Cameras
- Lights
- Shadow casters
- Skinned meshes

Large integer values are formatted through `RDT\_TextFormator`.

11.4 `SystemInfoView`

Displays the collected device, CPU, GPU, memory, and display information.

Memory is displayed using MB or GB formatting. Processor frequency is displayed using MHz or GHz formatting.

11.5 `AppInfoView`

Displays application metadata and runtime quality configuration.

The view formats development build state, target frame rate, and VSync state into readable text.

11.6 Tab navigation

`MonitorCanvasTabButton` sends tab selection events to the main canvas.

The canvas enables the selected panel and disables the previously active panel.

Available tabs:

- Graph
- Rendering
- System
- App

12. Formatting Utility

`RDT_TextFormator`

This static utility provides three formatting helpers.

`GetFormattedString(int value)`

Formats large integers using:

- `'K'` for thousands
- `'M'` for millions
- `'B'` for billions

Examples:

1,200 -> 1.2K

1,500,000 -> 1.5M

2,000,000,000 -> 2B

`GetMemoryString(int megabytes)`

Converts MB to GB when the value is at least 1024 MB.

`GetFrequencyString(int megahertz)`

Converts MHz to GHz when the value is at least 1000 MHz. Non-positive values return `'N/A'`.

13. Demo Stress Test

13.1 `StressTestManager`

The demo stress test instantiates a prefab at random positions inside a configurable box.

Features:

- Optional play-on-start
- Configurable prefab
- Optional spawn parent
- Maximum active instances
- Spawn interval
- Object lifetime
- Random rotation
- Random scale
- Scene-view spawn box gizmo
- Start, stop, clear, and stop-and-clear methods

Spawn behavior

The manager starts a coroutine that:

7. Removes destroyed objects from its tracking list.
8. Checks whether the maximum instance count has been reached.
9. Instantiates a prefab when capacity is available.
10. Waits for the configured interval.
11. Repeats while running.

The spawn position is generated in local box space and transformed into world space using the manager's transform.

Public methods

StartStressTest()

StopStressTest()

ClearInstances()

StopAndClear()

GetActiveInstanceCount()

IsRunning()

Stress test limitations

This utility intentionally uses `Instantiate` and `Destroy`. It is suitable for testing object creation/destruction and scene load, but it is not an object-pooling benchmark.

The active instance list is cleaned on the next spawn cycle or when the active count is requested.

14. Setup Guide

Step 1: Add the monitor prefab

Drag:

PerformanceMonitor/Prefab/[FPSMonitor].prefab

into the scene.

Step 2: Configure the monitor

Set the desired `FPSRate` on the `PerformanceMonitor` component.

Recommended initial values:

- `Hz\_10` for lower overhead
- `Hz\_25` for more responsive statistics
- `EveryFrame` for per-frame measurements when required

Step 3: Verify UI references

Ensure the prefab contains valid references for:

- FPS text fields
- Graph components
- Min/mid/max graph labels
- Rendering text fields
- System text fields
- Application text fields
- Tab buttons and canvas panels

Step 4: Run the scene

The monitor waits for its startup delay, initializes the calculators, and begins collecting data.

Step 5: Use the demo stress test

Add `StressTestManager` to a GameObject and assign:

- A test prefab
- Maximum instances
- Spawn interval
- Object lifetime
- Spawn box size

Enable `Play On Start` or call `StartStressTest()` from another component.

15. Data Flow

Unity runtime

|

v

PerformanceMonitor

1

+--> FPS Calculator

1 1

| v

| FPSSnapshot

1 1

| v

| FPSHistory

||

| v

| FPSStats

11

| v

| PerformanceMonitorCanvas

1 1

```
| +--> FPSView
```

| +--> FPS Graph

| +--> Frame-Time Graph

1

+--> RenderingInfoCalculator

1 1

| v

| RendererStats

||

1 v

| RenderingInfoView

1

+--> SystemStatsCalculator

| |

| v

| SystemStats

| |

| v

| SystemInfoView

|

+--> AppStatsCalculator

|

v

AppStats

|

v

AppInfoView

16. Performance and Maintenance Recommendations

High priority

12. ****Reuse the FPS sorting buffer.****

The current implementation allocates a new array every time FPS statistics are calculated.

13. ****Separate calculation and UI refresh rates.****

The graph refresh interval currently depends on the FPS calculation interval.

14. ****Validate profiler recorder availability.****

Profiler marker names can vary by Unity version and platform. Missing markers should be handled defensively.

15. ****Handle unsupported platform values.****

Some system information values can be zero or unavailable.

Medium priority

16. Cache scene object counts if the rendering panel does not need one-second updates.

17. Consider updating low 1% and low 10% metrics less frequently than basic FPS statistics.
18. Add explicit handling for all VSync count values.
19. Add public configuration properties or serialized settings for history size and refresh intervals.
20. Consider adding an enable/disable switch for each monitoring category.

Future features

- CPU timing
- GPU timing
- Memory monitoring
- Runtime command console
- Scene inspection
- Physics debugging
- AI/navigation debugging
- Object pooling stress-test mode
- CSV or JSON data export
- Runtime recording and replay
- Threshold-based warnings
- Configurable UI themes

17. Terminology

Term	Meaning
FPS	Frames per second
Frame time	Duration required to render a frame
Average FPS	Total samples divided by total measured frame time
Low 1% FPS	FPS equivalent calculated from the slowest 1% of samples
Low 10% FPS	FPS equivalent calculated from the slowest 10% of samples
Circular buffer	Fixed-size buffer that wraps around when full
Draw calls	Rendering submission calls reported by Unity's profiler
SetPass calls	Material/shader state changes reported by Unity's profiler
Batches	Rendering batches reported by Unity
VSync	Synchronization of frame presentation with display refresh
IL2CPP	Unity's C++-based scripting backend
Mono	Unity's Mono scripting backend

|---|---|

| FPS | Frames per second |

| Frame time | Duration required to render a frame |

| Average FPS | Total samples divided by total measured frame time |

| Low 1% FPS | FPS equivalent calculated from the slowest 1% of samples |

| Low 10% FPS | FPS equivalent calculated from the slowest 10% of samples |

| Circular buffer | Fixed-size buffer that wraps around when full |

| Draw calls | Rendering submission calls reported by Unity's profiler |

| SetPass calls | Material/shader state changes reported by Unity's profiler |

| Batches | Rendering batches reported by Unity |

| VSync | Synchronization of frame presentation with display refresh |

| IL2CPP | Unity's C++-based scripting backend |

| Mono | Unity's Mono scripting backend |

18. Known Limitations

- The monitor is currently implemented as a singleton and persists between scenes.

- The singleton does not explicitly destroy the duplicate object before continuing through the remainder of `Awake`; the duplicate may still execute subsequent initialization logic in that frame unless guarded.
- Low-percentile statistics are recalculated on every accepted FPS snapshot.
- The sorting buffer is reallocated in the current `FPSHistory` implementation.
- Scene-wide renderer, light, and skinned mesh enumeration can be expensive in large scenes.
- Profiler recorder metric names may require validation across Unity versions and platforms.
- The graph range is dynamically derived from current minimum and maximum values and may change as the data changes.
- The graph's refresh delay is coupled to the FPS calculation delay.
- The stress-test utility uses instantiate/destroy rather than pooling.
- System and GPU information availability varies by platform.

19. Summary

The current FPS Monitor module provides a complete initial runtime diagnostics workflow:

- It samples and aggregates frame performance.
- It stores snapshots in a circular buffer.
- It calculates summary and low-percentile statistics.
- It renders FPS and frame-time history graphs.
- It displays rendering, system, and application information.
- It includes a configurable prefab stress-test utility.

The most important production-hardening tasks are reducing allocations in `FPSHistory`, separating data collection from UI refresh, validating profiler markers, and testing the monitor across target platforms and scene sizes.